JT_DEV_B46_1.0

Full-Stack TypeScript App

1 Initial situation

1.1 In your previous projects, you practised front-end development with React and back-end development with Bun (you also learned some basics of PHP back-end). In this project, you will combine both into a single **full-stack TypeScript application**. You will build a React front-end that uses TanStack Router for routing, and a Bun back-end API using Elysia framework.

1.2 Your API will persist data in a SQL database with Bun.SQL and a PostgreSQL database. Your HTTP input validation will be done with Valibot.

1.3 The goal is to deliver a small but complete product: a web UI, a real API, and a database, all working together with strong typing, good error handling and separation of concerns.

1.4 You will create a simple time tracking application where users can log their daily work hours with a description of the tasks performed. To ensure simplicity, the application will not have authentication or user management: all entries are public and can be created, edited, viewed or deleted by anyone connected. All entries can be linked to a predefined list of projects and tagged with multiple predefined labels. The app should propose a clean system to clock in and out (and change clocked projects or labels). A table with pagination should list all entries. CRUD operations should be available to manage projects and labels. A statistics page should summarise total hours worked per project and per label (with time filters). The use of materialised views or similar techniques to optimise performance is encouraged.

2 Skills from the ICT training plan worked on

Skills ranked by importance of implementation in the project: consult the training plan.

2.1 `g5` : Implémenter les applications et les interfaces selon le concept en respectant les exigences de sécurité.

2.2 `c2` : Mettre en œuvre des modèles de données dans un dispositif de stockage de données numériques.

2.3 `a4` :Planifier les projets ICT et les tâches selon un modèle de procédure.

2.4 `a6` : Vérifier l'avancement des projets ICT et des tâches et en faire état selon le modèle de procédure.

2.5 `g6` : Vérifier la qualité et la sécurité des applications et des interfaces.

2.6 `h3` : Implémenter le processus de livraison des applications.

3 Deliverables

3.1 A GitHub repository containing the project.

3.2 A setup guide in the README that lets your trainer run the app locally (front-end + API + DB) out of the box.

4 Andragogic conditions

4.1 Work is carried out independently, on an individual basis.

4.2 You are allowed to help each other, share ideas and discuss solutions, but each trainee must provide their own implementation with their own code.

4.3 Apprentices have access to all the resources they need, particularly on the web.

4.4 Don't hesitate to ask AI for explanations, but avoid asking it for answers. The more you ask for answers, the less you learn.

4.5 **Maximum allowed time for the project: 12 full days (7.5 hours per day).** You must manage your time independently. On project beginning, inform your trainer of the start date and time.

5 Steps

5.1 Create a new GitHub Project from the TypeScript clock app template to your account (Use the « … » then « Make a copy » menu). Select your account to host the project and name the project with your GitHub pseudo, for example `TS-clock-app-<your-username>`. **Do not forget to tick the « Draft issues will be copied if selected » checkbox!**

5.2 Ensure you have Bun installed on your computer.

5.3 Ensure you have Docker installed on your computer to simplify running the database.

5.4 Setup your project by following instructions in the « project setup » task.

5.5 Implement all the tasks described in the project.

6 Resources

6.1 Bun: https://bun.sh/

6.2 Elysia documentation: https://elysiajs.com/

6.3 Valibot documentation: https://valibot.dev/

6.4 TanStack Router documentation: https://tanstack.com/router/latest

6.5 React documentation: https://react.dev/learn

6.6 SQL basics: https://sql.sh/

6.7 PostgreSQL: https://www.postgresql.org/docs/

6.8 You can use Tailwind CSS: https://tailwindcss.com/docs/installation

6.9 PostgreSQL Docker image: https://hub.docker.com/_/postgres

7 Evaluation criteria

7.1 The application runs locally (on first try) following the README instructions.

7.2 The database schema is defined with SQL schema DDL.

7.3 The API is implemented with Elysia and exposes the required CRUD endpoints.

7.4 Request validation uses Valibot and validation errors are returned in a clear, consistent JSON format.

7.5 The React app uses TanStack Router for navigation (file-based routes), and the main routes (list/create-edit/detail) work.

7.6 Front-end and back-end integration is correct (data persists and is displayed correctly).

7.7 The code is clean, typed, and follows conventions; there is no dead code.

7.8 Commits follow the conventional commits standard.

7.9 Back-end is structured with a feature-based architecture.

7.10 All commits are linted, formatted, and type-checked.

7.11 For each task there is a pull request, with a reviewer and a close keyword.

7.12 All the required features are implemented as described in the project tasks.

7.13 The README contains clear setup instructions that work on the first try.